

SQL Window Functions Intro

Deliverables: 12 queries, outputs, notes

Dataset: Northwind (SQLite) | Functions: ROW_NUMBER, RANK, DENSE_RANK, LAG | Revenue = UnitPrice x Quantity x (1 - Discount)

Long timestamps are shown as dates in Q1, Q2 and Q10. Output is limited with LIMIT where noted in the query.

Q1. Number every order per customer (ROW_NUMBER)

ROW_NUMBER. Orders numbered 1..n per customer by date; first 3 orders of 3 customers.

QUERY

```
SELECT CustomerID, OrderID, OrderDate,
       ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate, OrderID) AS order_seq
FROM Orders
WHERE CustomerID IN ('ALFKI', 'ANATR', 'ANTON')
ORDER BY CustomerID, order_seq
LIMIT 9;
```

OUTPUT

CustomerID	OrderID	OrderDate	order_seq
ALFKI	12671	2012-08-22	1
ALFKI	12078	2012-08-31	2
ALFKI	11908	2012-09-07	3
ALFKI	14208	2012-10-06	4
ALFKI	19569	2012-12-16	5
ALFKI	12696	2012-12-18	6
ALFKI	11789	2013-01-05	7
ALFKI	26469	2013-01-06	8
ALFKI	16617	2013-02-12	9

Note: ROW_NUMBER always gives unique numbers; OrderID is added as a tie-breaker so results are deterministic.

Q2. Each customer's first order (ROW_NUMBER + CTE)

ROW_NUMBER. Latest/first record per group, the classic dedupe pattern.

QUERY

```
WITH t AS (
  SELECT CustomerID, OrderID, OrderDate,
         ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate, OrderID) AS rn
  FROM Orders)
SELECT CustomerID, OrderID, OrderDate AS first_order_date
FROM t WHERE rn = 1
ORDER BY CustomerID
LIMIT 10;
```

OUTPUT

CustomerID	OrderID	first_order_date
ALFKI	12671	2012-08-22
ANATR	26158	2012-08-30
ANTON	11898	2012-07-13
AROUT	16993	2012-07-11
BERGS	13705	2012-08-07
BLAUS	19539	2012-11-02
BLONP	20776	2012-08-15
BOLID	18994	2012-07-13
BONAP	20949	2012-07-12
BOTTM	20106	2012-08-01

Note: Window functions can't go in WHERE, so wrap in a CTE/subquery and filter rn = 1.

Q3. Top 3 most expensive products per category (ROW_NUMBER)

ROW_NUMBER. Top-N per group.

QUERY

```
WITH t AS (  
    SELECT c.CategoryName, p.ProductName, p.UnitPrice,  
           ROW_NUMBER() OVER (PARTITION BY p.CategoryID ORDER BY p.UnitPrice DESC) AS rn  
    FROM Products p JOIN Categories c ON c.CategoryID = p.CategoryID)  
SELECT CategoryName, ProductName, UnitPrice, rn  
FROM t WHERE rn <= 3  
ORDER BY CategoryName, rn;
```

OUTPUT

CategoryName	ProductName	UnitPrice	rn
Beverages	Côte de Blaye	263.50	1
Beverages	Ipoh Coffee	46	2
Beverages	Chang	19	3
Condiments	Vegie-spread	43.90	1
Condiments	Northwoods Cranberry Sauce	40	2
Condiments	Sirop d'érable	28.50	3
Confections	Sir Rodney's Marmalade	81	1
Confections	Tarte au sucre	49.30	2
Confections	Schoggi Schokolade	43.90	3
Dairy Products	Raclette Courdavault	55	1
Dairy Products	Queso Manchego La Pastora	38	2
Dairy Products	Gudbrandsdalsost	36	3
Grains/Cereals	Gnocchi di nonna Alice	38	1
Grains/Cereals	Wimmers gute Semmelknödel	33.25	2
Grains/Cereals	Gustaf's Knäckebröd	21	3
Meat/Poultry	Thüringer Rostbratwurst	123.79	1
Meat/Poultry	Mishi Kobe Niku	97	2
Meat/Poultry	Alice Mutton	39	3
Produce	Manjimup Dried Apples	53	1
Produce	Rössle Sauerkraut	45.60	2
Produce	Uncle Bob's Organic Dried Pears	30	3
Seafood	Carnarvon Tigers	62.50	1
Seafood	Ikura	31	2
Seafood	Gravad lax	26	3

Note: With ties on price, ROW_NUMBER picks arbitrarily; Q5 shows how RANK handles that.

Q4. Rank products by revenue (RANK)

RANK. Overall revenue leaderboard.

QUERY

```
SELECT p.ProductName,  
       ROUND(SUM(od.UnitPrice*od.Quantity*(1-od.Discount)),2) AS revenue,  
       RANK() OVER (ORDER BY SUM(od.UnitPrice*od.Quantity*(1-od.Discount)) DESC) AS revenue_rank  
FROM "Order Details" od JOIN Products p ON p.ProductID = od.ProductID  
GROUP BY p.ProductID, p.ProductName  
ORDER BY revenue_rank  
LIMIT 10;
```

OUTPUT

ProductName	revenue	revenue_rank
Côte de Blaye	53,265,895.23	1
Thüringer Rostbratwurst	24,623,469.23	2
Mishi Kobe Niku	19,423,037.50	3
Sir Rodney's Marmalade	16,653,807.36	4
Carnarvon Tigers	12,604,671.88	5
Raclette Courdavault	11,216,410.70	6
Manjimup Dried Apples	10,664,768.65	7
Tarte au sucre	9,952,936.07	8
Ipoh Coffee	9,333,374.70	9

ProductName	revenue	revenue_rank
Rössle Sauerkraut	9,252,765.44	10

Note: Window functions are evaluated after GROUP BY, so they can wrap aggregates.

Q5. Rank products by price within category (RANK)

RANK. Ties share a rank, and the next rank is skipped.

QUERY

```
SELECT c.CategoryName, p.ProductName, p.UnitPrice,
       RANK() OVER (PARTITION BY p.CategoryID ORDER BY p.UnitPrice DESC) AS price_rank
FROM Products p JOIN Categories c ON c.CategoryID = p.CategoryID
WHERE c.CategoryName = 'Beverages'
ORDER BY price_rank
LIMIT 8;
```

OUTPUT

CategoryName	ProductName	UnitPrice	price_rank
Beverages	Côte de Blaye	263.50	1
Beverages	Ipoh Coffee	46	2
Beverages	Chang	19	3
Beverages	Chai	18	4
Beverages	Steeleye Stout	18	4
Beverages	Chartreuse verte	18	4
Beverages	Lakkalikööri	18	4
Beverages	Outback Lager	15	8

Note: If two items tie at rank 2, the next is rank 4 (gap).

Q6. RANK vs DENSE_RANK vs ROW_NUMBER side by side

Compare. The three ranking functions on tied data (Quantity per order line).

QUERY

```
SELECT OrderID, ProductID, Quantity,
       ROW_NUMBER() OVER w AS row_num,
       RANK() OVER w AS rnk,
       DENSE_RANK() OVER w AS dense_rnk
FROM "Order Details"
WHERE OrderID = 10248 OR OrderID = 10250
WINDOW w AS (ORDER BY Quantity DESC)
ORDER BY Quantity DESC, ProductID;
```

OUTPUT

OrderID	ProductID	Quantity	row_num	rnk	dense_rnk
10250	51	35	1	1	1
10250	65	15	2	2	2
10248	11	12	3	3	3
10250	41	10	5	4	4
10248	42	10	4	4	4
10248	72	5	6	6	5

Note: Same ORDER BY, different tie behavior. Named WINDOW clause avoids repeating the definition.

Q7. Top 3 customers per country by spend (DENSE_RANK)

DENSE_RANK. Top-N including ties, no rank gaps.

QUERY

```
WITH spend AS (  
  SELECT c.Country, c.CompanyName,  
         ROUND(SUM(od.UnitPrice*od.Quantity*(1-od.Discount)),2) AS total_spend  
  FROM Customers c  
  JOIN Orders o ON o.CustomerID = c.CustomerID  
  JOIN "Order Details" od ON od.OrderID = o.OrderID  
  GROUP BY c.CustomerID, c.Country, c.CompanyName),  
r AS (  
  SELECT *, DENSE_RANK() OVER (PARTITION BY Country ORDER BY total_spend DESC) AS dr  
  FROM spend)  
SELECT Country, CompanyName, total_spend, dr  
FROM r WHERE dr <= 3 AND Country IN ('USA', 'Germany', 'France')  
ORDER BY Country, dr;
```

OUTPUT

Country	CompanyName	total_spend	dr
France	Folies gourmandes	5,505,502.85	1
France	Spécialités du monde	5,298,269.65	2
France	Victuailles en stock	5,276,686.06	3
Germany	Morgenstern Gesundkost	5,349,089.59	1
Germany	QUICK-Stop	4,938,975.49	2
Germany	Ottilies Käseladen	4,911,223.40	3
USA	Hungry Coyote Import Store	5,698,023.67	1
USA	Let's Stop N Shop	5,462,198.02	2
USA	The Cracker Box	4,997,732.41	3

Note: DENSE_RANK is safer than RANK when you want 'top 3 distinct values'.

Q8. Monthly revenue with previous month (LAG)

LAG. Prior-period value beside the current one.

QUERY

```
WITH m AS (  
  SELECT substr(o.OrderDate,1,7) AS ym,  
         ROUND(SUM(od.UnitPrice*od.Quantity*(1-od.Discount)),2) AS revenue  
  FROM Orders o JOIN "Order Details" od ON od.OrderID = o.OrderID  
  WHERE o.OrderDate >= '2022-01-01' AND o.OrderDate < '2023-01-01'  
  GROUP BY 1)  
SELECT ym, revenue,  
       LAG(revenue) OVER (ORDER BY ym) AS prev_month_revenue  
FROM m ORDER BY ym;
```

OUTPUT

ym	revenue	prev_month_revenue
2022-01	3,591,353.72	NULL
2022-02	2,832,321.16	3,591,353.72
2022-03	3,222,005.69	2,832,321.16
2022-04	2,628,034.83	3,222,005.69
2022-05	3,719,270.11	2,628,034.83
2022-06	3,378,131.17	3,719,270.11
2022-07	3,358,177.30	3,378,131.17
2022-08	3,660,257.83	3,358,177.30
2022-09	2,606,755.13	3,660,257.83
2022-10	3,717,586.61	2,606,755.13
2022-11	3,542,815.27	3,717,586.61
2022-12	3,485,357.36	3,542,815.27

Note: First row has no predecessor, so LAG returns NULL.

Q9. Month-over-month growth % (LAG)

LAG. Trend analysis: change and % change.

QUERY

```
WITH m AS (  
  SELECT substr(o.OrderDate,1,7) AS ym,  
         SUM(od.UnitPrice*od.Quantity*(1-od.Discount)) AS revenue  
  FROM Orders o JOIN "Order Details" od ON od.OrderID = o.OrderID  
  WHERE o.OrderDate >= '2022-01-01' AND o.OrderDate < '2023-01-01'  
  GROUP BY 1)  
SELECT ym,  
       ROUND(revenue,2) AS revenue,  
       ROUND(revenue - LAG(revenue) OVER (ORDER BY ym),2) AS mom_change,  
       ROUND(100.0*(revenue - LAG(revenue) OVER (ORDER BY ym))  
         / LAG(revenue) OVER (ORDER BY ym),1) AS mom_pct  
FROM m ORDER BY ym;
```

OUTPUT

ym	revenue	mom_change	mom_pct
2022-01	3,591,353.72	NULL	NULL
2022-02	2,832,321.16	-759,032.56	-21.10
2022-03	3,222,005.69	389,684.53	13.80
2022-04	2,628,034.83	-593,970.86	-18.40
2022-05	3,719,270.11	1,091,235.28	41.50
2022-06	3,378,131.17	-341,138.94	-9.20
2022-07	3,358,177.30	-19,953.87	-0.60
2022-08	3,660,257.83	302,080.53	9.00
2022-09	2,606,755.13	-1,053,502.70	-28.80
2022-10	3,717,586.61	1,110,831.48	42.60
2022-11	3,542,815.27	-174,771.34	-4.70
2022-12	3,485,357.36	-57,457.91	-1.60

Note: Divide by the LAG value (the base), not the current one. NULLIF can guard against divide-by-zero.

Q10. Days between a customer's consecutive orders (LAG)

LAG. Reorder-gap / churn signal.

QUERY

```
SELECT CustomerID, OrderID, OrderDate,  
       LAG(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate, OrderID) AS prev_order_date,  
       CAST(julianday(OrderDate) - julianday(  
         LAG(OrderDate) OVER (PARTITION BY CustomerID ORDER BY OrderDate, OrderID)) AS INT) AS days_since_prev  
FROM Orders  
WHERE CustomerID IN ('ALFKI','ANATR')  
ORDER BY CustomerID, OrderDate, OrderID  
LIMIT 10;
```

OUTPUT

CustomerID	OrderID	OrderDate	prev_order_date	days_since_prev
ALFKI	12671	2012-08-22	NULL	NULL
ALFKI	12078	2012-08-31	2012-08-22	9
ALFKI	11908	2012-09-07	2012-08-31	6
ALFKI	14208	2012-10-06	2012-09-07	29
ALFKI	19569	2012-12-16	2012-10-06	71
ALFKI	12696	2012-12-18	2012-12-16	1
ALFKI	11789	2013-01-05	2012-12-18	18
ALFKI	26469	2013-01-06	2013-01-05	1
ALFKI	16617	2013-02-12	2013-01-06	36
ALFKI	16370	2013-03-25	2013-02-12	41

Note: PARTITION BY stops LAG from reaching into another customer's rows. julianday is SQLite-specific (use DATEDIFF elsewhere).

Q11. Product price change vs previous order line (LAG with offset/default)

LAG. LAG(col, offset, default).

QUERY

```
SELECT ProductID, OrderID, UnitPrice,
       LAG(UnitPrice, 1, UnitPrice) OVER (PARTITION BY ProductID ORDER BY OrderID) AS prev_price,
       UnitPrice - LAG(UnitPrice, 1, UnitPrice) OVER (PARTITION BY ProductID ORDER BY OrderID) AS price_change
FROM "Order Details"
WHERE ProductID = 11
ORDER BY OrderID
LIMIT 8;
```

OUTPUT

ProductID	OrderID	UnitPrice	prev_price	price_change
11	10248	14	14	0
11	10296	16.80	14	2.80
11	10327	16.80	16.80	0.00
11	10353	16.80	16.80	0.00
11	10365	16.80	16.80	0.00
11	10407	16.80	16.80	0.00
11	10434	16.80	16.80	0.00
11	10442	16.80	16.80	0.00

Note: Third argument replaces NULL for the first row, so the change becomes 0 instead of NULL.

Q12. Employee sales ranking + gap to leader (RANK + window aggregate)

RANK + LAG. Combines ranking with comparison to the row above.

QUERY

```
WITH s AS (
  SELECT e.FirstName || ' ' || e.LastName AS employee,
         ROUND(SUM(od.UnitPrice*od.Quantity*(1-od.Discount)),2) AS sales
  FROM Employees e
  JOIN Orders o ON o.EmployeeID = e.EmployeeID
  JOIN "Order Details" od ON od.OrderID = o.OrderID
  GROUP BY e.EmployeeID)
SELECT employee, sales,
       RANK() OVER (ORDER BY sales DESC) AS sales_rank,
       ROUND(sales - LAG(sales) OVER (ORDER BY sales DESC),2) AS gap_to_next_higher,
       ROUND(MAX(sales) OVER () - sales,2) AS gap_to_leader
FROM s ORDER BY sales_rank;
```

OUTPUT

employee	sales	sales_rank	gap_to_next_higher	gap_to_leader
Margaret Peacock	51,488,395.20	1	NULL	0.00
Steven Buchanan	51,386,459.10	2	-101,936.10	101,936.10
Janet Leverling	50,445,573.76	3	-940,885.34	1,042,821.44
Nancy Davolio	49,659,423.23	4	-786,150.53	1,828,971.97
Robert King	49,651,899.30	5	-7,523.93	1,836,495.90
Laura Callahan	49,281,136.81	6	-370,762.49	2,207,258.39
Michael Suyama	49,139,966.56	7	-141,170.25	2,348,428.64
Anne Dodsworth	49,019,678.44	8	-120,288.12	2,468,716.76
Andrew Fuller	48,314,100.77	9	-705,577.67	3,174,294.43

Note: LAG over a DESC ordering looks at the next-higher performer.

Summary Notes

Function	Ties	Gaps?	Use it for
ROW_NUMBER	Broken arbitrarily (unique)	No	Dedupe, first/latest row, exactly N rows
RANK	Same rank	Yes (1,2,2,4)	Competition-style leaderboards
DENSE_RANK	Same rank	No (1,2,2,3)	Top-N distinct values, tiers
LAG(col, n, default)	n/a	n/a	Previous row: MoM change, gaps between events

Interview questions

RANK vs DENSE_RANK? Both give tied rows the same rank. RANK skips the following numbers (1,2,2,4); DENSE_RANK does not (1,2,2,3).

When use LAG? When a row needs a value from an earlier row in an ordered set: period-over-period growth, time between events, price changes. It replaces a self-join. LEAD is the forward-looking twin.

Pitfalls

Window functions cannot be filtered in WHERE, so use a CTE. Always add a tie-breaker to ORDER BY. Always PARTITION BY when comparing within an entity.